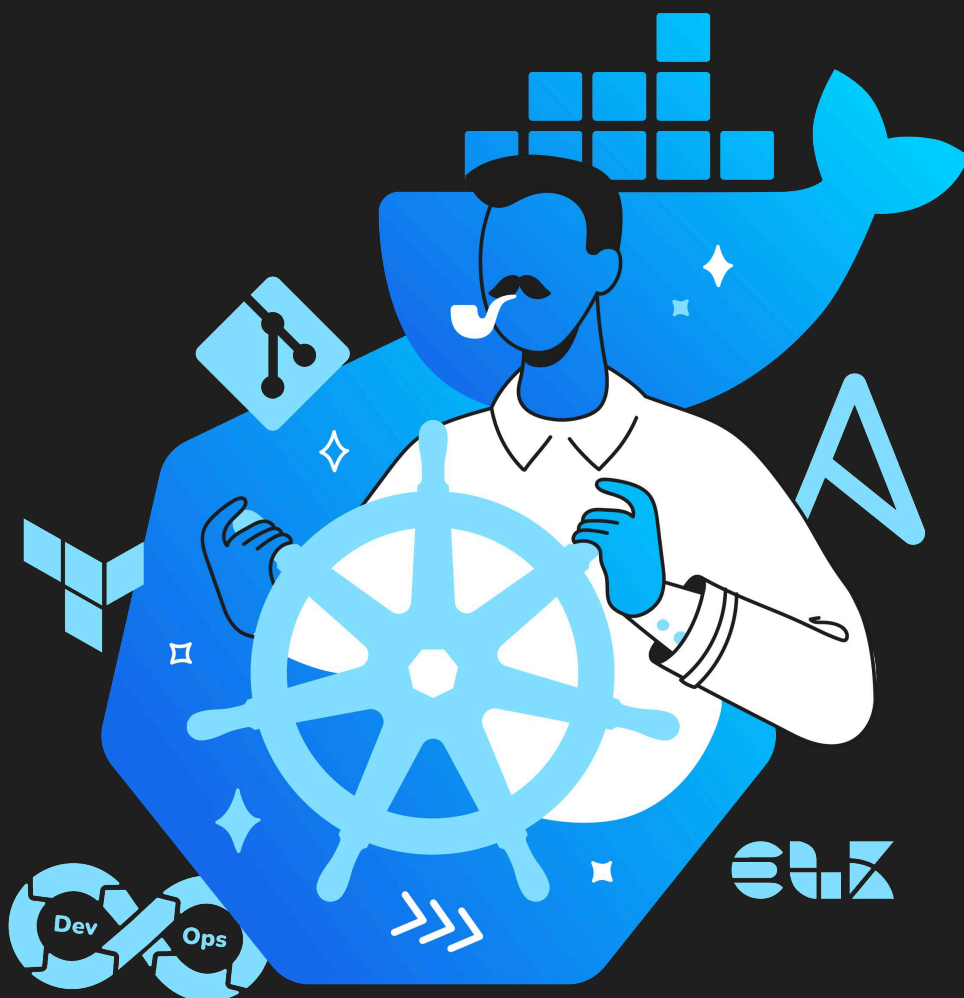


Методический материал

РАБОТА С МНОГОЭТАПНОЙ СБОРКОЙ ОБРАЗОВ DOCKER (MULTISTAGE)

Урок 6.





Для выполнения этого задания нам потребуется проект "Hello-docker", написанный на языке Golang. Данный проект представляет из себя легковесный веб-сервер, работающий на порту 8080 и отправляющий приветствие при обращении к нему.

Рассмотрим Dockerfile проекта:

```
$ cat Dockerfile
```

```
filipp@user-IdeaPad-3-15IAU7:~/hello-docker$ cat Dockerfile
FROM golang:alpine AS builder
WORKDIR /src
COPY go.mod .
COPY go.sum .
RUN go mod download
COPY main.go .
RUN go build

FROM alpine:3.15
WORKDIR /app
COPY --from=builder /src/hello-docker /app/hello-docker
ENTRYPOINT ["/app/hello-docker"]
```

Dockerfile содержит две инструкции FROM (FROM golang:alpine и FROM alpine:3.15), разделяя тем самым процедуру сборки на два этапа. Первый этап снабжен меткой (AS builder). Во втором этапе мы используем результат сборки первого этапа через обращение к метке (COPY --from=builder). Переходим в терминал и перемещаемся в директорию с проектом, где находится Dockerfile для многоэтапной (Multistaging) сборки.



Выполняем команду на сборку:

```
$ docker build . -t hello-world-multistage
```

```
fllpp@user-IdeaPad-3-15IAU7:~/hello-docker$ docker build . -t hello-world-multistage
[+] Building 20.4s (16/16) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 279B                                0.0s
=> [internal] load metadata for docker.io/library/alpine:3.15     2.7s
=> [internal] load metadata for docker.io/library/golang:alpine   2.7s
=> [internal] load .dockerignore                                   0.0s
=> => transferring context: 2B                                       0.0s
=> [stage-1 1/3] FROM docker.io/library/alpine:3.15@sha256:19b4bcc4f60e99dd5ebdca0cbce22c503bbcff1975  1.0s
=> => resolve docker.io/library/alpine:3.15@sha256:19b4bcc4f60e99dd5ebdca0cbce22c503bbcff197549d7e19d  0.0s
=> => sha256:19b4bcc4f60e99dd5ebdca0cbce22c503bbcff197549d7e19dab4f22254dc864 1.64kB / 1.64kB  0.0s
=> => sha256:6a0657acfe760bd9e293361c9b558e90e7d740ed8dffca823d17098a4ffddf5 520B / 520B    0.0s
=> => sha256:32b91e3161c8fc2e3baf2732a594305ca5093c82ff4e0c9f6ebbd2a879468e1d 1.47kB / 1.47kB  0.0s
=> => sha256:d078792c4f9122259f14b539315bd92cbd9490ed73e08255a08689122b143108 2.83MB / 2.83MB  0.9s
=> => extracting sha256:d078792c4f9122259f14b539315bd92cbd9490ed73e08255a08689122b143108  0.1s
=> [builder 1/7] FROM docker.io/library/golang:alpine@sha256:7772cb5322baa875edd74705556d08f0eeca7b9 10.5s
=> => resolve docker.io/library/golang:alpine@sha256:7772cb5322baa875edd74705556d08f0eeca7b9c4b536775  0.0s
=> => sha256:7772cb5322baa875edd74705556d08f0eeca7b9c4b5367754ce3f2f00041ccee 10.29kB / 10.29kB  0.0s
=> => sha256:3077e12cda6deb78a9eba8eba0b6b4efe6f9c17295a18e3883cc5797d1688acb 1.92kB / 1.92kB  0.0s
=> => sha256:dce68b1cd2290b5ece1593ada1a0ebb7962c2d2d820d98342cddcd0e2f1495e4 2.08kB / 2.08kB  0.0s
=> => sha256:cfdff1cf77cc238c3b8e5663c583e181f4a737027e840204038f02071b9c7faf 294.90kB / 294.90kB  0.4s
=> => sha256:f1d296901bdc593d88a0813bb00eef0974b68222cba6add046b831c006a1c68c 78.94MB / 78.94MB  7.5s
=> => extracting sha256:cfdff1cf77cc238c3b8e5663c583e181f4a737027e840204038f02071b9c7faf  0.0s
=> => sha256:efda6a9ec0ed27775d0887572a23317fed2c90e7dc2dbe7ce0dfdbddc8ae41f6 126B / 126B    0.5s
=> => sha256:4f4fb700ef54461cfa02571ae0db9a0dc1e0cdb5577484a6d75e68dc38e8acc1 32B / 32B      0.7s
=> => extracting sha256:f1d296901bdc593d88a0813bb00eef0974b68222cba6add046b831c006a1c68c  2.8s
=> => extracting sha256:efda6a9ec0ed27775d0887572a23317fed2c90e7dc2dbe7ce0dfdbddc8ae41f6  0.0s
=> => extracting sha256:4f4fb700ef54461cfa02571ae0db9a0dc1e0cdb5577484a6d75e68dc38e8acc1  0.0s
=> [internal] load build context                                   0.0s
=> => transferring context: 1.44kB                                    0.0s
=> [stage-1 2/3] WORKDIR /app                                     0.0s
=> [builder 2/7] WORKDIR /src                                     0.2s
=> [builder 3/7] COPY go.mod .                                    0.0s
=> [builder 4/7] COPY go.sum .                                    0.0s
=> [builder 5/7] RUN go mod download                             1.2s
=> [builder 6/7] COPY main.go .                                    0.0s
=> [builder 7/7] RUN go build                                     5.5s
=> [stage-1 3/3] COPY --from=builder /src/hello-docker /app/hello-docker  0.0s
=> exporting to image                                             0.0s
=> => exporting layers                                              0.0s
=> => writing image sha256:1ed4f0e7093a1036ee6cb17f550aa0e20a816ecaa953b3708239afa2d5184b6a  0.0s
=> => naming to docker.io/library/hello-world-multistage          0.0s
```

Запускаем контейнер на основе собранного образа:

```
hello-world-multistage: $ docker run hello-world-multistage
```

```
fllpp@user-IdeaPad-3-15IAU7:~/hello-docker$ docker run hello-world-multistage
```



Откроем новое окно терминала и выполним команду инспектирования bridge сети, узнаем присвоенный запущенному контейнеру IP-адрес:

```
$ docker network inspect bridge
```

```
hello-docker : docker
flllpp@user-IdeaPad-3-15IAU7:~/hello-docker$ docker run hello-world-multistage
[]

hello-docker : bash
{
  "Name": "bridge",
  "Id": "71855f574129062516d70dc81f98a9d372f6e51ef1ed1198a757ef4a9fe82cb7",
  "Created": "2025-04-14T10:09:22.00613694+03:00",
  "Scope": "local",
  "Driver": "bridge",
  "EnableIPv4": true,
  "EnableIPv6": false,
  "IPAM": {
    "Driver": "default",
    "Options": null,
    "Config": [
      {
        "Subnet": "172.17.0.0/16",
        "Gateway": "172.17.0.1"
      }
    ]
  },
  "Internal": false,
  "Attachable": false,
  "Ingress": false,
  "ConfigFrom": {
    "Network": ""
  },
  "ConfigOnly": false,
  "Containers": {
    "bea7ca11fb198b69f4481950ffdddf4f556a7b6c203728dbc0a53e390203d00ad": {
      "Name": "gifted_newton",
      "EndpointID": "903e6992a58d5c36e59a42f4f0dacecfb6e8b78058389952d7a42fb373646a3c",
      "MacAddress": "26:92:b8:15:52:10",
      "IPv4Address": "172.17.0.2/16",
      "IPv6Address": ""
    }
  }
}
```



Выполним запрос к нашему веб-серверу в контейнере, чтобы получить в ответ приветствие, убедившись в корректности работы:

```
$ curl http://<ip-adress>:8080
```

```
hello-docker : docker
filipp@user-IdeaPad-3-15IAU7:~/hello-docker$ docker run hello-world-multistage
[]

hello-docker : bash
filipp@user-IdeaPad-3-15IAU7:~/hello-docker$ curl 172.17.0.2:8080
Hello docker!
```

Выполним команду по сборке промежуточного образа (1-этап в данной многоэтапной сборке), используя обращение по метке (метка Builder первой инструкции FROM в Dockerfile):

```
$ docker build --target builder . -t hello-world-multistage-intermediate
```

```
filipp@user-IdeaPad-3-15IAU7:~/hello-docker$ docker build --target builder . -t hello-world-multistage-intermediate
[+] Building 2.7s (12/12) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 279B                                0.0s
=> [internal] load metadata for docker.io/library/golang:alpine   2.4s
=> [internal] load .dockerignore                                   0.0s
=> => transferring context: 2B                                       0.0s
=> [builder 1/7] FROM docker.io/library/golang:alpine@sha256:7772cb5322baa875edd74705556d08f0eeca7b9c 0.0s
=> [internal] load build context                                   0.0s
=> => transferring context: 81B                                       0.0s
=> CACHED [builder 2/7] WORKDIR /src                                0.0s
=> CACHED [builder 3/7] COPY go.mod .                              0.0s
=> CACHED [builder 4/7] COPY go.sum .                              0.0s
=> CACHED [builder 5/7] RUN go mod download                       0.0s
=> CACHED [builder 6/7] COPY main.go .                            0.0s
=> CACHED [builder 7/7] RUN go build                              0.0s
=> exporting to image                                              0.2s
=> => exporting layers                                              0.2s
=> => writing image sha256:bfdd6649763e6cfe8b1780672ec6afc562d3922ef124cdfb23ef3bd3ea68f8a8 0.0s
=> => naming to docker.io/library/hello-world-multistage-intermediate 0.0s
```



Выполним команду для просмотра собранных образов на каждом из двух этапов Multistage-сборки:

```
$ docker images
```

```
filipp@user-IdeaPad-3-15IAU7:~/hello-docker$ docker images
REPOSITORY              TAG          IMAGE ID      CREATED        SIZE
hello-world-multistage   latest       1ed4f0e7093a  7 minutes ago  13.8MB
hello-world-multistage-intermediate latest       bfdd6649763e  7 minutes ago  370MB
docker-python-alpine-3   latest       7ba9788432ed  18 minutes ago 409MB
docker-python-alpine-2   latest       c3346f9ecdec  20 minutes ago 412MB
docker-python-1          latest       f285e872b452  5 days ago    1.02GB
hello-world              latest       74cc54e27dc4  2 months ago  10.1kB
```

Мы получили с вами легковесный образ на финальном этапе многоэтапной (Multistaging) сборки. В данный Docker image, где использовался базовый образ Alpine, вошел только бинарник, собранный на 1-м этапе многоэтапной сборки. Образ 1-го этапа (Builder) может быть переиспользован для последующих сборок golang-приложений в Docker или удален за ненадобностью.